

Week 6: CI Pipeline Implementation(CO2)

- Create Jenkins pipeline scripts
- Automate build and test processes
- Configure email notifications

To create a Jenkins CI pipeline that automatically:

1. Gets source code from GitHub
 2. Builds the Maven project
 3. Runs tests
 4. Packages the application
 5. Sends email notifications for successful or failed builds
-

1. Software Required

JDK

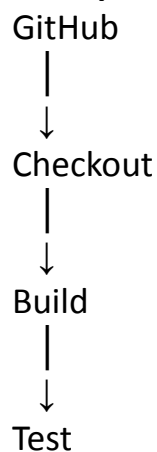
Git

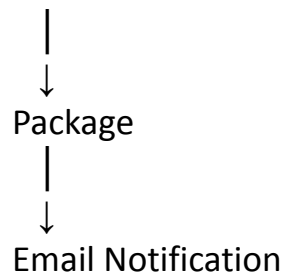
Maven

Jenkins

GitHub account

Pipeline we are going to implement





2. Verify Java Installation

Open **Command Prompt**.

Command

`java -version`

Description

This command checks whether Java is installed and displays the installed Java version.

Expected output will look similar to:

`java version "..."`

3. Verify Maven Installation

Command

`mvn -version`

Description

This command checks whether Maven is installed and configured correctly.

Expected output:

Apache Maven 3.9.x

Java version: ...

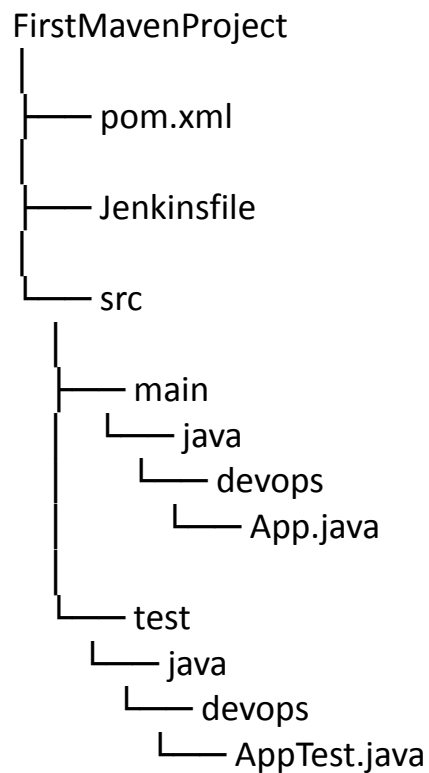
Maven home: ...

If mvn is not recognized, Maven's bin directory needs to be added to the Windows PATH.

4. Create a Maven Project

A simple Maven project can be used for this experiment.

The project structure should be:



5. Create Java Program

Create:

src/main/java/devops/App.java

Code

```
package devops;

public class App {

    public static void main(String[] args) {

        System.out.println("Hello DevOps");

    }
}
```

Description

This is the Java application that Jenkins will compile and build.

6. Create Maven Configuration

Create:

pom.xml

Code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>

  <artifactId>FirstMavenProject</artifactId>

  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>

</project>
```

Description

pom.xml is the main configuration file of a Maven project.

It tells Maven:

- Project name
 - Group ID
 - Version
 - Java version
 - Build information
-

7. Test the Maven Project Manually

Open Command Prompt in the project directory.

For example:

```
cd C:\Users\Prani\Desktop\FirstMavenProject
```

Description

cd changes the current directory to your Maven project directory.

Check the Project

Command

```
dir
```

Description

This displays the files and folders in the project.

You should see something similar to:

```
pom.xml
```

```
src
```

8. Clean the Project

Command

```
mvn clean
```

Description

clean removes previously generated Maven build files, especially the target directory.

It gives us a fresh build environment.

9. Compile the Project

Command

```
mvn compile
```

Description

This command compiles the Java source code.

The process is:

```
App.java
```

↓
Java Compiler
↓
Compiled .class file
If successful:

BUILD SUCCESS

10. Run the Tests

Command

mvn test

Description

This command executes the test cases present in the project.

If all tests pass:

BUILD SUCCESS

If a test fails:

BUILD FAILURE

11. Package the Application

Command

mvn package

Description

This command:

Compile

↓

Test

↓

Package

The final package is normally created inside:

target

For example:

target\FirstMavenProject-1.0-SNAPSHOT.jar

12. Upload the Project to GitHub

First initialize Git.

Command

```
git init
```

Description

This creates a Git repository in the current project directory.

Add Project Files

Command

```
git add .
```

Description

. means **all files in the current project**.

This command stages the project files for committing.

Create a Commit

Command

```
git commit -m "Initial Maven project"
```

Description

This saves the current version of the project in Git with the message:

Initial Maven project

Connect to GitHub

Create a repository on GitHub, for example:

FirstMavenProject

Then add the repository URL.

Command

```
git remote add origin https://github.com/USERNAME/FirstMavenProject.git
```

Description

This connects your local Git repository to your GitHub repository.

Replace:

USERNAME

with your GitHub username.

Push the Project

Command

git branch -M main

Description

This changes the current branch name to:

main

Command

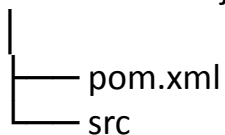
git push -u origin main

Description

This uploads your project from the local computer to GitHub.

Now GitHub should contain:

FirstMavenProject



13. Create the Jenkinsfile

Now comes the **main part of Week 6**.

Inside the project folder, create a file named exactly:

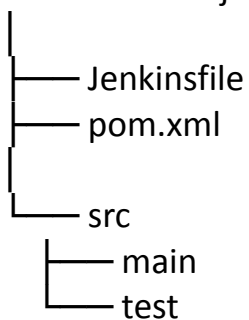
Jenkinsfile

Do not create:

Jenkinsfile.txt

The structure should now be:

FirstMavenProject



14. Write the Jenkins Pipeline

For your **Windows Jenkins**, use bat.

Jenkinsfile

```
pipeline {  
  
    agent any  
  
    stages {  
  
        stage('Checkout') {  
            steps {  
                checkout scm  
            }  
        }  
  
        stage('Build') {  
            steps {  
                bat 'mvn clean compile'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                bat 'mvn test'  
            }  
        }  
  
        stage('Package') {  
            steps {  
                bat 'mvn package'  
            }  
        }  
    }  
  
    post {  
  
        success {  
            echo 'CI Pipeline completed successfully!'  
        }  
    }  
}
```

```
        failure {  
            echo 'CI Pipeline failed!'  
        }  
    }  
}
```

15. Understand the Jenkinsfile

pipeline

```
pipeline {
```

Description

This tells Jenkins that we are defining a **Pipeline**.

agent any

```
agent any
```

Description

It tells Jenkins that the pipeline can execute on any available Jenkins agent.

For a basic laboratory setup, this is sufficient.

16. Checkout Stage

```
stage('Checkout') {
```

```
    steps {
```

```
        checkout scm
```

```
    }
```

```
}
```

Description

This stage obtains the source code from the GitHub repository configured for the Jenkins job.

The process is:

GitHub

↓
Jenkins
↓
Source Code

17. Build Stage

```
stage('Build') {  
  
    steps {  
  
        bat 'mvn clean compile'  
  
    }  
}
```

Description

This stage builds the Java application.

The command:

mvn clean compile
performs:

clean

↓

Remove old build files

↓

compile

↓

Compile Java source code

Because we are using **Windows**, we use:

bat

For Linux Jenkins, the equivalent would normally be:

sh 'mvn clean compile'

18. Test Stage

```
stage('Test') {
```

```
steps {  
  
    bat 'mvn test'  
  
}  
}
```

Description

This stage executes the automated test cases.

Test cases



Execute



Pass / Fail

If tests pass:

BUILD SUCCESS

If tests fail:

BUILD FAILURE

19. Package Stage

Stage

```
stage('Package') {  
  
    steps {  
  
        bat 'mvn package'  
  
    }  
}
```

Description

This stage creates the final application package.

The package will normally appear in:

target

20. Post Section

```
post {  
  
    success {  
  
        echo 'CI Pipeline completed successfully!'  
  
    }  
  
    failure {  
  
        echo 'CI Pipeline failed!'  
  
    }  
}
```

Description

post executes after the pipeline finishes.

If the pipeline succeeds:

success
is executed.

If the pipeline fails:

failure
is executed.

21. Add Jenkinsfile to Git

After creating the Jenkinsfile:

Command

```
git add Jenkinsfile
```

Description

This adds the Jenkinsfile to the Git staging area.

Command

```
git commit -m "Added CI pipeline"
```

Description

This commits the Jenkins pipeline configuration.

Command

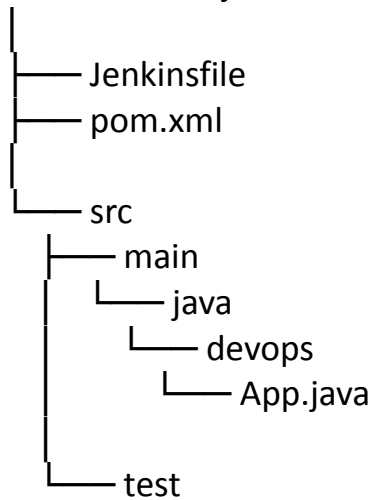
git push

Description

This uploads the Jenkinsfile to GitHub.

Your GitHub repository should now look like:

FirstMavenProject



22. Create Jenkins Pipeline Job

Open Jenkins.

Usually:

<http://localhost:8080>

Then:

Jenkins



New Item



Enter name



Pipeline



OK

For example, name the job:

FirstMaven-CI

23. Configure Pipeline from GitHub

In the Jenkins job:

Pipeline



Definition



Pipeline script from SCM

Select:

SCM → Git

Enter your GitHub repository URL:

<https://github.com/USERNAME/FirstMavenProject.git>

Select branch:

*/main

For:

Script Path

enter:

Jenkinsfile

Then:

Save

24. Execute the Pipeline

On the Jenkins job page:

FirstMaven-CI



Build Now

Description

Build Now manually starts the Jenkins pipeline.

Jenkins executes:

Checkout



Build



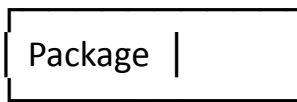
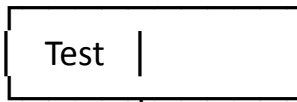
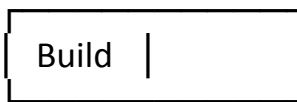
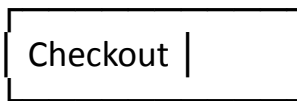
Test



Package

25. Observe the Pipeline

You should see stages such as:



SUCCESS

26. View Console Output

Click:

Build Number



Console Output

You should see Maven commands being executed.

At the end:

Finished: SUCCESS

Description

This confirms that Jenkins successfully completed the CI pipeline.

27. Configure Email Notification

Now we implement the third requirement of Week 6:

Configure email notifications.

First install Jenkins' **Email Extension Plugin** if it is not already installed.

Go to:

Manage Jenkins



Plugins



Available plugins

Search:

Email Extension Plugin

Install it.

28. Configure SMTP

Go to:

Manage Jenkins



System

Find:

Extended E-mail Notification

For Gmail, configure the SMTP server according to Google's current SMTP requirements.

Typically:

SMTP Server:

smtp.gmail.com

Use the appropriate SMTP port/security setting and authenticate using a Jenkins-managed credential. **Do not put your normal Gmail password into the Jenkinsfile.**

If using a Google account with 2-Step Verification, an **App Password** may be required.

29. Add Email to Jenkinsfile

After configuring email, modify the post section.

Example:

```
post {  
  
    success {  
  
        emailext(  
            subject: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER}",  
            body: "The Jenkins CI pipeline completed successfully.",  
            to: "your-email@example.com"  
        )  
    }  
  
    failure {  
  
        emailext(  
            subject: "FAILURE: ${env.JOB_NAME} #${env.BUILD_NUMBER}",  
            body: "The Jenkins CI pipeline failed. Please check the Jenkins console  
output.",  
            to: "your-email@example.com"  
        )  
    }  
}
```

Description

emailext sends an email using Jenkins' Email Extension Plugin.

For successful execution:

success



Email

For failed execution:

failure



Email

For a real setup, configure recipients and credentials securely through Jenkins rather than hard-coding secrets.

30. Push the Updated Jenkinsfile

Command

```
git add Jenkinsfile
```

Description

Stages the updated Jenkinsfile.

Command

```
git commit -m "Added email notification"
```

Description

Creates a Git commit containing the email notification configuration.

Command

```
git push
```

Description

Uploads the updated Jenkinsfile to GitHub.

31. Test Successful Pipeline

In Jenkins:

FirstMaven-CI



Build Now

Pipeline:

Pipeline:

Checkout



Build

↓
Test
↓
Package
↓
SUCCESS
↓
Email

You should receive an email similar to:

SUCCESS: FirstMaven-CI #1

32. Test Pipeline Failure

Now intentionally introduce an error in App.java.

For example:

```
System.out.println("Hello DevOps"
```

There is a missing:

```
);
```

Now save the file.

Command

```
git add .
```

Description

Stages the modified files.

Command

```
git commit -m "Test pipeline failure"
```

Description

Commits the intentionally incorrect code.

Command

```
git push
```

Description

Pushes the incorrect code to GitHub.

33. Jenkins Detects the Error

Run the pipeline again:

Build Now

Jenkins performs:

Checkout



Build



Compilation Error



FAILURE



Failure Email

Console output will show a compilation error and eventually:

Finished: FAILURE

You should receive a failure email.

34. Correct the Error

Change the program back to:

```
package devops;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Hello DevOps");
```

```
    }
```

```
}
```

Then:

```
git add .
```

Description

Stages the corrected code.

```
git commit -m "Fixed compilation error"
```

Description

Commits the corrected version.

```
git push
```

Description

Uploads the corrected code to GitHub.

Run Jenkins again.

Expected result:

Checkout



Build



Test



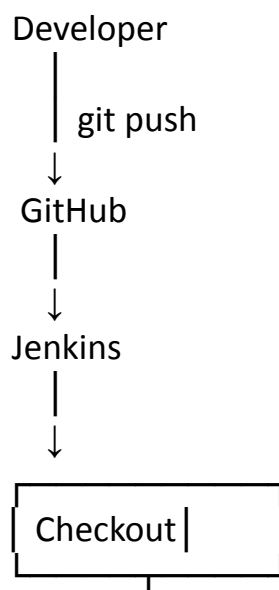
Package

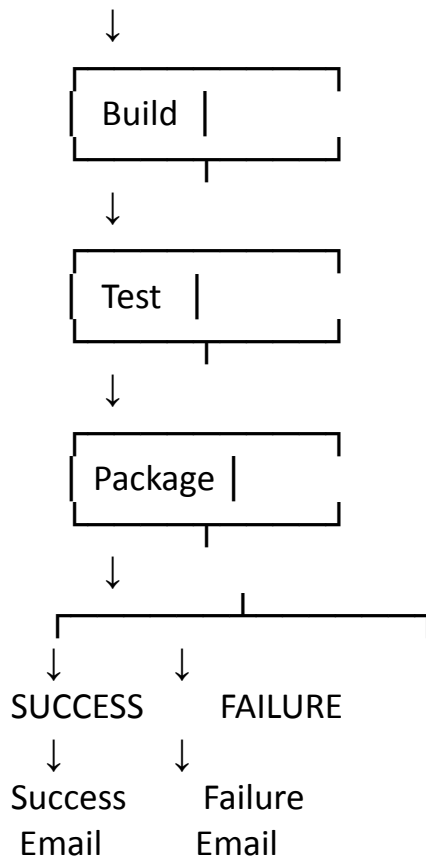


SUCCESS

35. Final CI Pipeline

Your complete experiment now works like this:





36. Commands to Remember

For your students, the important commands can be remembered in this order:

1. java -version

↓
Check Java

2. mvn -version

↓
Check Maven

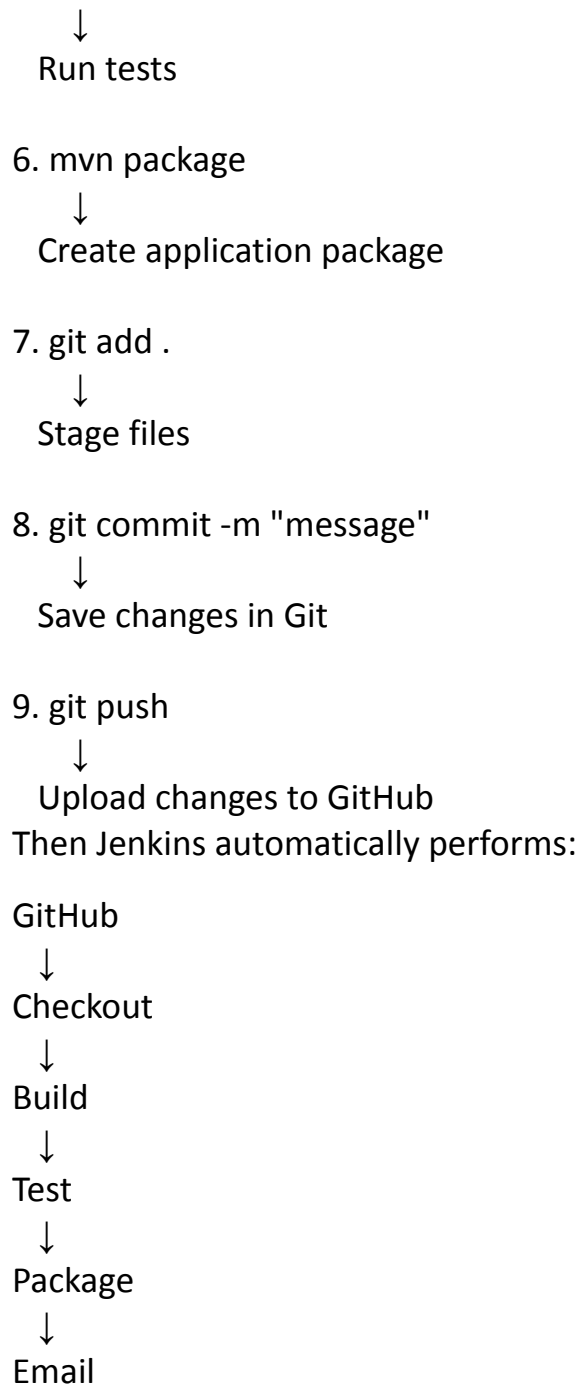
3. mvn clean

↓
Remove old build files

4. mvn compile

↓
Compile Java program

5. mvn test



Result: The Jenkins Continuous Integration pipeline was successfully implemented. The pipeline automatically checked out the source code from GitHub, built the Maven project, executed test cases, packaged the application, and generated email notifications for successful and failed builds.

Most important concept to explain in the viva
Pipeline = a sequence of automated stages.

For this experiment:

Checkout → Build → Test → Package → Notify

The **main significance** is that whenever new code is integrated, Jenkins can automatically build and test it, detect errors early, and inform the developer of the result.

Bottom of Form